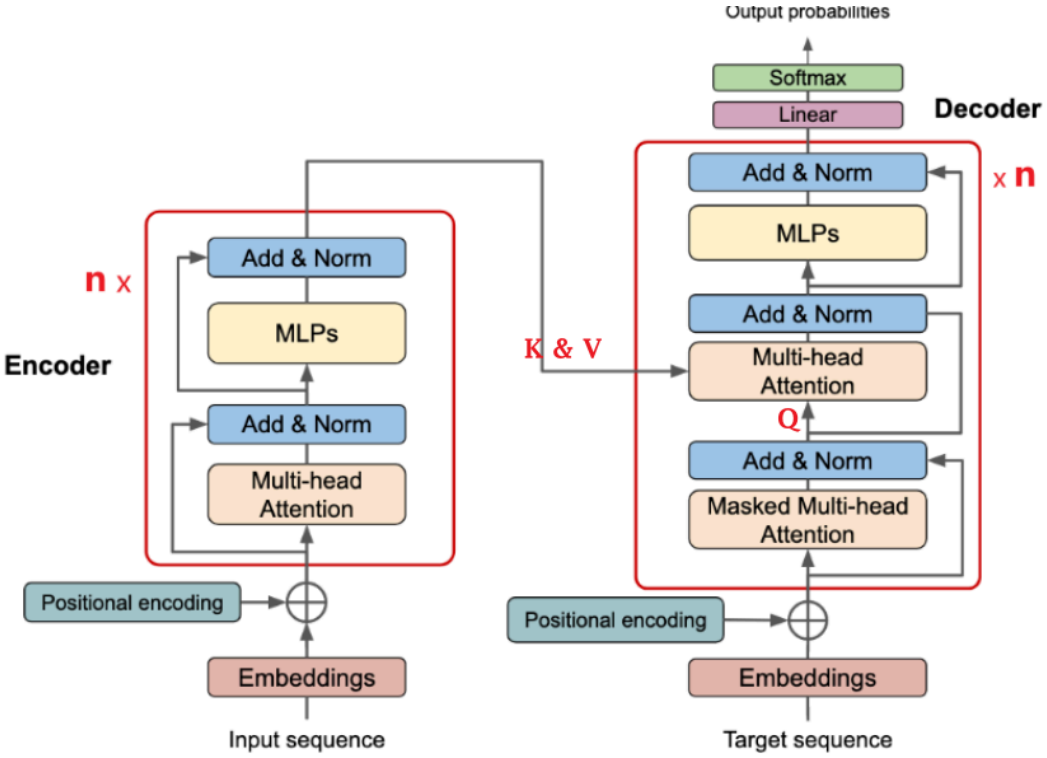
Transformer -- 基于Attention的神经网络架构

自注意力具有**并行计算**和**最短的最大路径长度**这两个优势, 因此使用自注意力来设计深度架构是很有吸引力的

Transfrom模型[Vaswani et al.,2017]完全基于注意力机制, 没有任何的Conv和rnn层

Transformer最初应用于在文本数据上的Seq2Seq学习, 但现在已经推广到各种现代的深度学习中, 例如语言, 视觉, 语音和强化学习领域.

一. Transformer 模型



1. 编码器

编码器由多个相同的层叠加而成, 每个层都有两个子层(sublayer),

1. 第一个子层是**多头自注意力**汇聚
2. 第二个子层是**基于位置的前馈网络**

计算**编码器**的自注意力时, query, key和values都来自前一个编码器层的输出.

(1)关于ADD&Norm

对于序列中任何位置的任何输入$x \in R^d$, 都满足$\text{sublayer}(x) \in R^d$, 以便**残差连接**满足$x + \text{sublayer}(x) \in R^d$

在残差连接的加法Add计算后, 紧接着就是**应用层规范化(layer normalization)**.

2. 解码器

解码器也是有多个相同的层叠加而成, 并且层中使用了残差连接和层规范化
每个解码器层除了两个子层 *Multi-head和FFN* 外, 还在这两个子层之间插入了第三个子层, 称为**编码器-解码器注意力(encoder-decoder attention)** 层

1. *在编码器-解码器注意力*中, **查询来自前一个解码层的输出, 而键和值来自整个编码器的输出**
2. *在解码器自注意力*中, 查询和键和值都来自**上一个解码器层的输出**

掩蔽:解码器中的每个位置只能考虑该位置之前的所有位置, 这种**掩蔽(masked)** 注意力保留了*自回归(auto-regressive)* 属性, 确保预测**仅依赖于已生成的输出词元**

二. 编码器-解码器架构

机器翻译是*seq2seq*型的核心问题, 其输入和输出都是长度可变的序列.
针对这些问题设计包含两个主要组件的架构:

1. **编码器(encoder)**: 他接受一个长度可变的序列作为输入, 并将其转换为具有固定形状的编码状态
2. **解码器(decoder)**: 他将固定形状的编码状态映射到长度可变的序列.

三. 基于位置的神经网络(FFN)

前馈神经网络(Feed-Forward Network)一般是两层MLP, 他能够扩大词向量的维度
输入X的形状为 *(批量大小, 时间步数或序列长度, 隐单元数或特征维度)* 将被一个两层的感知机转换为 *(批量大小, 时间步, ffn_num_outputs)*

```python
class PositionWiseFFN(nn.Module):
    """基于位置的前馈网络"""
    def __init__(self, ffn_num_input, ffn_num_hiddens, ffn_num_outputs, **kwargs):
        super(PositionWiseFFN, self).__init__(**kwargs)
        self.dense1 = nn.Linear(ffn_num_input, ffn_num_hiddens)
        self.relu = nn.ReLU()
        self.dense2 = nn.Linear(ffn_num_hiddens, ffn_num_outputs)

    def forward(self, X):
        return self.dense2(self.relu(self.dense1(X)))
```

四. 残差连接和层归一化 (add&norm)

1. 层归一化

> 规范化/归一化(Normalization)都是为了使每一层的输入分布更加稳定

层规范化(LayerNorm) 其实和 *batch norm* 的目标相同, 但层规范化是**基于特征维度**进行规范化, 每个token都独立归一化. [BathNorm沿batch维度, 计算batch的均值和方差]
在自然语言处理任务重(输入通常是变长序列), *BatchNorm*通常不如*LayerNorm*的效果好

```python
>>> ln = nn.LayerNorm(normalized_shape=2)
>>> bn = nn.BatchNorm1d(num_features=2)
>>> X = torch.tensor([[1, 2], [2, 3]], dtype=torch.float32)
>>> # 在训练模式下计算X的均值和方差
>>> print('layer norm:', ln(X), '\nbatch norm:', bn(X))

layer norm: tensor([[-1.0000,  1.0000], # 每个barch(行)单独进行norm
                    [-1.0000,  1.0000]], grad_fn=<NativeLayerNormBackward0>)
batch norm: tensor([[-1.0000, -1.0000],  # 沿batch维度(列)进行norm
                    [ 1.0000,  1.0000]], grad_fn=<NativeBatchNormBackward0>)
```

2. AddNorm实现

现在使用残差连接和层规范化实现AddNorm类:

```python
class AddNorm(nn.Module):
    """残差连接后进行层规范化"""
    def __init__(self, normalized_shape, dropout, **kwargs):
        super(AddNoem, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout) # dropout正则化用于减少对上一子层输出的依赖, 提高泛化
        self.ln = nn.LayerNorm(normalized_shape)

    def forward(self, X, Y):
        """
        X : residual connection
        Y : output from last Mutil-head sublayer
        """
        return self.ln(self.dropout(Y) + X)
```

五. 编码器

编码器负责将输入序列"编码"成一种连续的,富态语义的表示, 相当于将源语言压缩成机器易于处理的中间形式

先是输入嵌入(Embedding), 然后是位置编码, 然后经过n个**编码器层**(多头自注意力 + 基于位置前馈神经网络)

1. ENcoderBlock

先实现编码器中的一个层. 下面的EncoderBlock类包含两个子层: [多头注意力和基于位置的前馈网络. 每个子层都有AddNorm]

```python
class EncoderBlock(nn.Module):
    """Transformer编码器块"""
    def __init__(self, key_size, query_size, value_size,
    num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
    num_heads, dropout, use_bias=False, **kwargs):
        super(EncoderBlock, self).__init__(**kwargs)
        self.attention = MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens,
            num_heads, dropout, use_bias
        )
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(
            ffn_num_input, ffn_num_hiddens, num_hiddens
        )
        self.addnorm2 = AddNorm(norm_shape, dropout)

    def forward(self, X, valid_lens):
        Y = self.addnorm1(X, self.attention(X, X, X, valid_lens))
        return self.addnorm2(Y, self.ffn(Y))
```

应用:

```python
X = torch.ones((2, 100, 24))
valid_lens = torch.tensor([3, 2])
encoder_blk = EncoderBlock(24, 24, 24, 24, [100, 24], 24, 48, 8, 0.5)
encoder_blk.eval()
encoder_blk(X, valid_lens).shape

# torch.Size([2, 100, 24])
```

2. TransformerEncoder

下面实现的Transformer编码器, 包含编码器前期的embedding和位置编码嵌入. 代码中堆叠了 `num_layer` 个 `EncoderBlock` 的实例

```python
class Encoder(nn.Module):
    """基本的编码器接口"""
    def __init__(self, **kwargs):
        super(Encoder, self).__init__(**kwargs)

    def forward(self, X, *args):
        raise NotImplementedError

class TransformerEncoder(Encoder):
    """Transformer编码器"""
    def __init__(self, vocab_size, key_size, query_size,value_size,
    num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
    num_heads, num_layers, dropout, use_bias=False, **kwargs):
    super(TransformerEncoder, self).__init__(**kwargs)
    self.num_hiddens = num_hiddens

    self.embedding = nn.Embedding(vocab_size, num_hiddens, dropout)
    self.pos_encoding = PositionalEncoding(num_hiddens, dropout)
    self.blks = nn.Sequential()
    for i in range(num_layers):
        self.blks.add_module("block"+str(i)),
        EncoderBlock(key_size, query_size, value_size,
        num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens, num_heads, dropout, use_bias)

    def forward(self, X, valid_lens, *args):
        # 因为位置编码值在-1和1之间，
        # 因此嵌入值乘以嵌入维度的平方根进行缩放，
        # 然后再与位置编码相加。
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self.attention_weights = [None] * len(self.blks)
        for i, blk in enumerate(self.blks):
            X = blk(X, valid_lens)
            self.attention_weight[i] = blk.attention.attention.attention_weights

        return X
```

应用:

```
encoder = TransformerEncoder(
    200, 24, 24, 24, 24, [100, 24], 24, 48, 8, 2, 0.5)
encoder.eval()
encoder(torch.ones((2, 100), dtype=torch.long), valid_lens).shape

# torch.Size([2, 100, 24])
```

六. 解码器

在`DecoderBlock`类中实现的每个层都包含了三个子层:

1. **掩蔽多头自注意力**
   在生成第t个词时, 解码器只能看到位置1到$t-1$的词, 不能看到未来词, 因此注意力计算时需要掩蔽(mask)未来位置, 即对QK_T矩阵中未来位置的分数设置为$-\infty$
2. **"编码器-解码器"注意力**/交叉注意力(cross-attention)
   此子层将解码器的当前表示作为查询Q, 编码器的输出 H 作为键K和值V，让解码器关注输入序列的相关部分
3. **基于位置的前馈网络**
   该层和编码器的FFN相同, 进一步变换特征

1.pytorch实现

分别实现单个编码器层`DecoderBlock`和编码器整体`TransformerDecoder`, 源代码在transformer.py

七. 具体训练过程